

El Paradigma Funcional

Introducción

El **paradigma funcional** es un paradigma de programación declarativa, basado en funciones matemáticas y el cálculo lambda (λ -calculus).

Se obtiene un nivel de abstracción mayor que en el paradigma imperativo ya que el programador se enfoca en qué problema debe resolver, y no en especificar un algoritmo que lo resuelva.

Las variables representan valores, no locaciones de memoria.

La ejecución consiste en la evaluación de funciones.

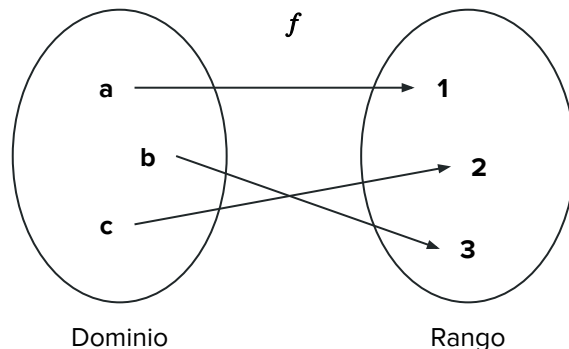
Se utiliza **pattern matching** para controlar el flujo de ejecución.

Lenguajes más alejados de la arquitectura von Neumann.

Introducción :: Funciones Matemáticas

Una función matemática es un mapeo de los elementos de un conjunto (dominio) a elementos de otro conjunto (rango).

- No dependen de valores externos a la función, sólo de los valores de sus argumentos.
- No tienen efectos colaterales.
- Siempre mapean un valor del dominio a un mismo valor del rango.
- Variables representan valores, y no pueden cambiar durante la evaluación de la función.



Introducción :: Cálculo Lambda

El **cálculo lambda** (**λ -calculus**) es un modelo de computación formal diseñado para investigar la definición de funciones, su aplicación y el concepto de recursión.

- Fue desarrollado por Alonzo Church a lo largo de la década de 1930.
- Computacionalmente equivalente a las máquinas de Turing.
- Utiliza **expresiones lambda** (*lambda expressions*), que especifican los parámetros y el mapeo de una función sin nombre.

$identidad(x) = x$ puede ser expresado como $x \rightarrow x$

El nombre de la función no es relevante.

$x, y \rightarrow x + y$ expresa la misma función que $u, v \rightarrow u + v$

El nombre de los argumentos no es relevante.

$x, y \rightarrow x + y$ puede expresarse como $x \rightarrow (y \rightarrow x + y)$

Una función con varios argumentos puede ser expresada como funciones de un único argumento.

Introducción :: Cálculo Lambda

Ejemplo

La siguiente función lambda recibe un parámetro y computa el cubo del mismo.

$$\lambda.x.x * x * x$$

$$(\lambda.x.x * x * x)(3) \Rightarrow 27$$

Ejemplo

La siguiente función lambda calcula la suma de sus argumentos.

$$\lambda.x.(\lambda.y.x + y)$$

$$\lambda.x.(\lambda.y.x + y)(3)(5) \Rightarrow (\lambda.y.3 + y)(5) \Rightarrow 8$$

Las expresiones lambda **sólo pueden tomar un argumento**, pero pueden retornar otras funciones.

Propiedades de los Lenguajes Funcionales Puros

Los siguientes conceptos resultan **fundamentales en el paradigma funcional** y en los lenguajes funcionales puros:

- Semántica por valores
- Funciones primitivas y funciones de orden superior
- Currying
- Transparencia referencial
- Evaluación perezosa
- Polimorfismo
- Pattern matching

Semántica por Valores

En el paradigma imperativo, una variable es una abstracción de una locación de memoria, donde se almacena un valor resultante de la evaluación de una expresión.

En el paradigma funcional, **una variable es una abstracción de un valor.**

Una variable quedará ligada a un único valor durante toda la ejecución del programa, y no podrá redefinirse.

Tabla de Referencias

x = 1

x	1
---	---

Semántica por Valores

En el paradigma imperativo, una variable es una abstracción de una locación de memoria, donde se almacena un valor resultante de la evaluación de una expresión.

En el paradigma funcional, **una variable es una abstracción de un valor**.

Una variable quedará ligada a un único valor durante toda la ejecución del programa, y no podrá redefinirse.

Tabla de Referencias

`x = 1`

...

`z1 = x + 1`

x	1
z1	2

Semántica por Valores

En el paradigma imperativo, una variable es una abstracción de una locación de memoria, donde se almacena un valor resultante de la evaluación de una expresión.

En el paradigma funcional, **una variable es una abstracción de un valor**.

Una variable quedará ligada a un único valor durante toda la ejecución del programa, y no podrá redefinirse.

Tabla de Referencias

```
x = 1
...
z1 = x + 1
...
x = 10
```

x	1
z1	2
x	10

Semántica por Valores

En el paradigma imperativo, una variable es una abstracción de una locación de memoria, donde se almacena un valor resultante de la evaluación de una expresión.

En el paradigma funcional, **una variable es una abstracción de un valor**.

Una variable quedará ligada a un único valor durante toda la ejecución del programa, y no podrá redefinirse.

Tabla de Referencias

`x = 1`

...

`z1 = x + 1`

...

`x = 10`

...

`z2 = x + 1`

x	1
z1	2
x	10
z2	11

No existe una locación de memoria
asociada a una variable.

Funciones

Los lenguajes funcionales puros brindan dos tipos de funciones: **funciones primitivas** y **funciones de orden superior** (formas funcionales).

Las **funciones primitivas** son las funciones básicas que brinda el lenguaje para manipular los objetos.

- Operadores aritméticos
- Operadores de comparación
- Manipulación de estructuras

Ejemplos

`add x y`

`mul x y`

`equal x y`

`head list`

`tail list`

`append x list`

`concat list1 list2`

Funciones

Los lenguajes funcionales puros brindan dos tipos de funciones: **funciones primitivas** y **funciones de orden superior** (formas funcionales).

Las **formas funcionales** nos permiten crear funciones más complejas a partir de las funciones primitivas. Pueden tomar una función como entrada y retornar una función como salida.

Forma Funcional	Propósito	Ejemplo
Map o Apply	Aplicar una función a todos los elementos de una secuencia	<pre>cuadrado x = mul x x apply(cuadrado, [1, 2, 3, 4]) ⇒ [1, 4, 9, 16]</pre>
Filter	Filtrar elementos de una secuencia que verifican una función dada	<pre>par x = equal mod(x, 2) 0 filter(par, [1, 2, 3, 4]) ⇒ [2, 4]</pre>
Composición	Aplicar una función sobre el resultado de otra función	<pre>f x = add x 1 g x = mul x 2 h w = f(g(w)) = add (mul w 2) 1 h 3 = f(g(3)) = add (mul 3 2) 1</pre>

Transparencia Referencial

El resultado de una función con **transparencia referencial** no depende de ningún elemento externo, sólo depende de sus argumentos. Los argumentos son exclusivamente “modo in”.

Dados los mismos valores de entrada, una función con transparencia referencial siempre produce el mismo resultado, independientemente del momento en que se invoque.

Contar con transparencia referencial facilita la comprensión de los programas, al igual que su verificación y mantenimiento.

$x = 1$

...

$f1\ y = y + 1$

...

f1 x



$x = 10$

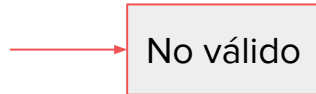
...

f1 x



...

f2 y = x + y



Evaluación Perezosa

Los lenguajes funcionales puros utilizan evaluación perezosa, de forma que **las expresiones se evalúan sólo cuando es necesario**.

Los resultados de las evaluaciones se almacenan, para poder reutilizarlos en caso de necesitar evaluar nuevamente la misma expresión.

`cuadrado x = mul x x`

`cuadrado 3! = mul 3! 3! = mul 6 3! = mul 6 6 = 36`



La evaluación perezosa resulta adecuada en los lenguajes funcionales puros porque las funciones gozan de **transparencia referencial**.

Currying

Dado que el cálculo lambda solo admite funciones unarias, la evaluación de múltiples parámetros se logra mediante la técnica de **currying**.

Esta técnica permite transformar una función de varios argumentos en una cadena de funciones anidadas, donde cada una toma un único argumento y devuelve la siguiente función de la secuencia.

Cada función retornará otra función que toma los restantes argumentos de la función inmediata anterior.

$$f\ x\ y\ z = x + y + z$$



$$f'\ x \Rightarrow g\ y\ z$$



$$g'\ y \Rightarrow h\ z$$

La técnica de currying nos permite
**crear funciones a partir de la
aplicación parcial** de otras funciones.

Currying

Se puede definir funciones con más de un argumento como una aplicación en cadena de funciones con un único argumento.

Cada aplicación parcial define una nueva función con un argumento menos. La técnica de currying nos permite **crear funciones a partir de la aplicación parcial** de otras funciones.

Función	Expresión de tipo
<pre>filter f [] = [] filter f [h::t] = if f h then h::filter f t else filter f t</pre>	<pre>(A -> boolean) -> (List of A -> List of A)</pre>
<pre>pares lista = filtrar_pares lista filtrar_pares = filter es_par</pre>	<pre>filtrar_pares : List of int -> (List of int -> List of int) pares : List of int -> List of int</pre>
<pre>vocales lista = filter es_vocal lista</pre>	<pre>List of char -> List of char</pre>

Pattern matching

Los lenguajes funcionales utilizan **pattern matching** como mecanismo de control de flujo.

El pattern matching permite establecer una relación entre un valor y un patrón, lo cual **determina el flujo de ejecución**. El patrón corresponde con la definición de una función, y el valor corresponde con una expresión.

```
f x = add x 1
```

```
g x = mul x 2
```

```
h w = f(g(w)) = add (mul w 2) 1
```



```
h 3
```

```
= f(g(3))
```

```
= add g(3) 1
```

```
= add (mul 3 2) 1
```

Además permite descomponer valores estructurados en sus partes constituyentes.

Función	Expresión de tipo
<pre>length [] = 0 length [h::t] = 1 + length t</pre>	<pre>List of A -> int</pre>

Polimorfismo

Una función se dice **polimórfica** si se comporta de manera uniforme sobre un conjunto de tipos de datos.

Los lenguajes funcionales puros implementan **polimorfismo paramétrico** de manera natural infiriendo **expresiones de tipo que contienen variables de tipo**.

Función	Expresión de tipo
<code>map f [] = []</code> <code>map f [h::t] = (f h)::map f t</code>	<code>(A -> A) -> (List of A -> List of A)</code>
<code>cuadrados lista = map cuadrado lista</code>	<code>List of int -> List of int</code>
<code>mayusculas lista = map to_upper lista</code>	<code>List of char -> List of char</code>

Caso de Estudio: Haskell

Introducción

Haskell es un lenguaje puro del paradigma funcional que cuenta con algunas características interesantes:

- Lenguaje compilado **fuertemente tipado** con **tipado estático**.
- Utiliza un avanzado mecanismo de inferencia de tipos
- Utiliza evaluación perezosa (lazy evaluation) por defecto.
- Polimorfismo paramétrico de manera natural.

```
removeDuplicates :: Eq a => [a] -> [a]
removeDuplicates [] = []
removeDuplicates (x:xs) =
    let rest = removeDuplicates xs
    in if x `elem` rest then rest
       else x : rest

lista :: [Int]
lista = removeDuplicates [1, 1, 1]
```

Introducción

Nombre de la función

Firma de tipo

Parámetro

Resultado

remove_duplicates
se define por
casos

Constructor para alcance
local (*let-in*)

```
removeDuplicates :: Eq a -> [a] -> [a]
removeDuplicates [] = []
removeDuplicates (x:xs) =
  let rest = removeDuplicates xs
  in if elem x rest then rest
     else x : rest

ta :: [Int]
ta = removeDuplicates [1, 1, 1]
```

Llamada recursiva

Expresión condicional
con evaluación perezosa

Identificadores y Ligaduras

La declaración de variables (valores) en el nivel principal no utiliza ninguna palabra reservada; simplemente se asocian mediante el signo igual (=).

Para declaraciones locales se usan bloques como `let` o `where`.

```
tasaDeImpuesto :: Double
tasaDeImpuesto = 0.21

nombreSistema :: String
nombreSistema = "Sistema de facturación"

calcularPrecio :: Double -> Double
calcularPrecio precioBase =
    let impuesto = precioBase * tasaDeImpuesto
        precioFinal = precioBase + impuesto
    in
        precioFinal
```

Una variable es puramente una
abstracción de un valor.

El valor ligado al identificador es
inmutable y no puede cambiar
durante la ejecución del programa.

Identificadores y Ligaduras

La declaración de funciones no requiere una palabra reservada. Simplemente se escribe el identificador (el nombre de la función) seguido de sus parámetros, y este queda ligado a una expresión mediante el signo `=`.

```
cuadrado :: Num a => a -> a
cuadrado x = x * x

areaRectangulo :: Num a => a -> a -> a
areaRectangulo base altura = base * altura

areaRectanguloTpl :: Num a => (a, a) -> a
areaRectanguloTpl (base, altura) = base * altura

areaConBaseCinco :: Double -> Double
areaConBaseCinco = areaRectangulo 5.0
```

Las funciones tienen un sólo argumento. Las funciones con más de un argumento son soportadas como funciones curried, aunque también pueden simularse usando tuplas.

Identificadores y Ligaduras

Es posible definir un identificador local a una expresión usando la expresión `let...in`.

Además del `let ... in`, Haskell brinda la cláusula `where` para definir identificadores locales al nivel de una función. Esta permite escribir primero la expresión principal y dejar las declaraciones locales para el final, lo que a menudo hace que el código sea más fácil de leer.

```
areaCilindroLet :: Double -> Double -> Double
areaCilindroLet r h =
    let areaLateral = 2 * pi * r * h
        areaBases = 2 * pi * r^2
    in areaLateral + areaBases
```

```
areaCilindroWhere :: Double -> Double -> Double
areaCilindroWhere r h = areaLateral + areaBases
    where
        areaLateral = 2 * pi * r * h
        areaBases = 2 * pi * r^2
```

Cuando termina la evaluación de la expresión, las ligaduras de la declaración se descartan.

Sistema de Tipos

El lenguaje provee los tipos básicos Int (entero de precisión fija), Integer (entero de precisión arbitraria), Float, Double, Char, String y Bool.

Además, brinda tuplas y listas como tipos estructurados.

- Una tupla permite agrupar una cantidad fija de valores que pueden ser de distintos tipos
- Una lista es una colección ordenada de elementos del mismo tipo.

```
edad :: Int
edad = 35

temperatura :: Float
temperatura = 23.5

piPreciso :: Double
piPreciso = 3.141592653589793

mensaje :: String
mensaje = "Lenguajes de Programación"

esActivo :: Bool
esActivo = True

estudiante :: (String, Int, Double)
estudiante = ("Ana", 22, 8.75)

notas :: [Int]
notas = [8, 9, 7, 10]
```

Sistema de Tipos

El lenguaje permite al usuario definir sus propios tipos de datos a través de la palabra reservada `data`. El usuario debe definir el nombre del nuevo tipo (del lado izquierdo del `=`) y los constructores de datos que pertenecen al mismo (a la derecha).

Los constructores de datos definen las distintas formas, variantes o estructuras que pueden tomar los valores del tipo.

```
data Dia = Lunes | Martes | Miercoles | Jueves | Viernes | Sabado | Domingo
```

```
data Figura = Circulo Double | Rectangulo Double Double
```

```
data Arbol a = Hoja | Nodo a (Arbol a) (Arbol a)
```

```
data Maybe a = Nothing | Just a
```

```
area :: Figura -> Double  
area (Circulo r) = pi * r * r  
area (Rectangulo b h) = b * h
```

Sistema de Tipos

Un componente importante en el lenguaje son las **clases de tipos** (type classes). Una clase de tipos define un conjunto de operaciones (funciones) que un tipo de dato debe soportar para ser considerado una "instancia" de esa clase.

```
iguales :: Eq a => a -> a -> Bool
iguales x y = x == y
```

```
class Eq a where
  (==) :: a -> a -> Bool
  (/=) :: a -> a -> Bool
```

Para ser instancia de Eq, un tipo **a** debe brindar las operaciones == y /= (con la expresión de tipos que indica Eq)

```
data Foo = Foo {x :: Integer, str :: String}

instance Eq Foo where
  (Foo x1 str1) == (Foo x2 str2) = (x1 == x2) && (str1 == str2)
```

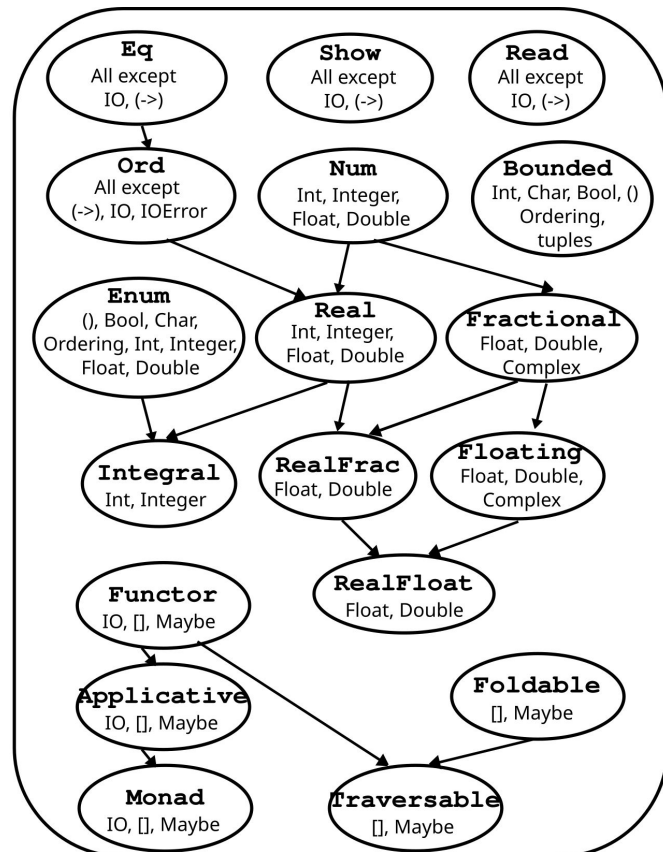
Sistema de Tipos

Un componente importante en el lenguaje son las **clases de tipos** (type classes).

Una clase de tipos define un conjunto de operaciones (funciones) que un tipo de dato debe soportar para ser considerado una "instancia" de esa clase.

A su vez, una clase de tipos puede heredar de otra (estableciendo mayores restricciones a los tipos que sean instancia).

```
class (Eq a) => Ord a where
  compare      :: a -> a -> Ordering
  (<), (<=), (>=), (>) :: a -> a -> Bool
  max, min     :: a -> a -> a
```



Sistema de Tipos

Haskell **no permite la sobrecarga de funciones de manera directa**, porque el algoritmo de inferencia de tipos debe poder resolver las expresiones de tipos de todas las expresiones de un programa.

```
data Circulo = MkCirculo Double
data Rectangulo = MkRectangulo Double Double

area :: Circulo -> Double
area (MkCirculo radio) = pi * radio ^ 2

area :: Rectangulo -> Double
area (MkRectangulo base altura) = base * altura
```



Sin embargo, **podemos utilizar Type Classes para definir comportamiento sobrecargado**.

El usuario puede definir funciones sobrecargadas que operen de distintas maneras dependiendo del tipo de dato que reciban, siempre y cuando el tipo implemente la clase correspondiente (por ejemplo, la clase Num para operaciones numéricas, Eq para verificar igualdad, o Show para convertir a texto).

Sistema de Tipos

Podemos utilizar Type Classes para definir comportamiento sobrecargado.

```
class Medible a where
  area :: a -> Double

data Circulo = MkCirculo Double
data Rectangulo = MkRectangulo Double Double

instance Medible Circulo where
  area (MkCirculo radio) = pi * radio ^ 2

instance Medible Rectangulo where
  area (MkRectangulo base altura) = base * altura

calcularCosto :: Medible a => a -> Double -> Double
calcularCosto figura precioPorMetro = area figura * 2
```

Para ser instancia de Medible, un tipo **a** debe brindar una función **area**

Círculo y Rectángulos son dos tipos definidos por el usuario

Círculo y Rectángulo brindan la función **area** con algoritmos diferentes

Sistema de Tipos

El lenguaje **no soporta expresiones mixtas ni incluye coerciones**.

Cualquier operación que involucre tipos diferentes requiere que el programador realice una conversión explícita utilizando funciones provistas por el lenguaje (como `fromIntegral` para pasar de tipos enteros a tipos de punto flotante).

```
cantidad :: Int
cantidad = 5

precio = 12.5

totalCorrecto :: Float
totalCorrecto = fromIntegral cantidad * precio

totalCorrectoEntero :: Int
totalCorrectoEntero = cantidad * round precio
```

Sistema de Tipos :: Inferencia de Tipos

Una parte fundamental del lenguaje es su mecanismo de inferencia de tipos. El tipo asociado a un identificador puede ser inferido a partir del contexto en el que aparece el identificador.

- El tipo de una variable es inferido a partir del valor al cual fue ligada y a partir de las operaciones en las que interviene.
- El tipo de una función es inferido a partir del tipo de sus argumentos y el tipo del resultado.

```
estado = "Activo"           -- estado :: String

total = 20                  -- total :: Double
mitad = total / 2           -- mitad :: Double

esVocal letra = letra `elem` "aeiou" -- esVocal :: Char -> Bool

sumarValores x y = x + y    -- sumarValores :: Num a => a -> a -> a

iguales x y = x == y        -- iguales :: Eq a => a -> a -> Bool

esSucesor x y = iguales (x + 1) y -- esSucesor :: (Eq a, Num a) => a -> a -> Bool
```

Sistema de Tipos :: Inferencia de Tipos

Si las operaciones son genéricas, el mecanismo de inferencia asignará el **tipo polimórfico más general posible**, añadiendo automáticamente restricciones de Clases de Tipos.

```
sumarValores x y = x + y           -- sumarValores :: Num a => a -> a -> a
iguales x y = x == y               -- iguales :: Eq a => a -> a -> Bool
esMayorAlDoble x y = x > (y + y) -- esMayorAlDoble :: (Ord a, Num a) => a -> a -> Bool
```

El compilador nunca elige un tipo concreto (como `Int` o `Float`) si no hay nada en el código que lo fuerce a hacerlo. Siempre va a preferir la versión polimórfica restringida.

```
sumarCinco = sumarValores 5        -- sumarCinco :: Integer -> Integer
sumarCincoF = sumarValores 5.0     -- sumarCincoF :: Double -> Double
```

Funciones

Las funciones en Haskell son consideradas ciudadanos de primera clase (first-class citizens).

De esta manera, una función puede ser pasada como parámetro de otra función (lo que da lugar a las funciones de orden superior), y también ser retornada como resultado de la evaluación de otra función.

```
duplicar :: Int -> Int
duplicar x = x * 2
```

```
aplicarDosVeces :: (a -> a) -> a -> a
aplicarDosVeces f x = f (f x)
```

```
crearSumador :: Int -> (Int -> Int)
crearSumador n = \x -> x + n
```

```
ejemplo1 = aplicarDosVeces duplicar 5
ejemplo2 = (crearSumador 10) 5
```

```
-- Resultado: 20
-- Resultado: 15
```

Función lambda



Funciones

El tipo de las funciones polimórficas es un **politipo**, esto es, un tipo que contiene **variables de tipo**.

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (x:xs) = f x : map f xs
```

Politipo con dos variables de tipo

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ [] = []
filter p (x:xs)
  | p x = x : filter p xs
  | otherwise = filter p xs
```

```
ejemploFilter :: [Int] -> [Int]
ejemploFilter = filter (> 5) [2, 5, 8, 10]

ejemploFilterStrings = filter (\s -> length s > 3)
                           ["hi", "haskell", "ai"]

contarLetras lista = map length lista
```

Las variables de tipo se instanciarán con tipos particulares al momento de utilizar la función polimórfica.

Funciones

Conceptualmente, en Haskell todas las funciones tienen un único argumento. Las funciones que requieren más de un argumento son soportadas de manera nativa como **funciones curried**. Las funciones curried permiten definir nuevas funciones a partir de **aplicaciones parciales**.

Alternativamente se pueden simular múltiples parámetros mediante el uso de **tuplas**. Aquí la aplicación parcial no es posible.

El lenguaje utiliza **evaluación perezosa**. Se posterga la evaluación de cualquier expresión hasta el momento en que su resultado es necesario para que el programa pueda continuar.

```
sumaCurried :: Int -> Int -> Int
sumaCurried x y = x + y

sumarDiez :: Int -> Int
sumarDiez = sumaCurried 10

sumaTupla :: (Int, Int) -> Int
sumaTupla (x, y) = x + y
```

Control de Flujo :: Pattern Matching

Una de las principales características de Haskell es el uso extensivo de **pattern matching**. El mecanismo permite (1) desestructurar datos para ligar variables a valores específicos dentro de una estructura más compleja (como listas, tuplas o tipos de datos definidos por el usuario), y (2) controlar el flujo de la ejecución de un programa.

```
sumarVectores :: (Double, Double) -> (Double, Double) -> (Double, Double)
```

```
sumarVectores (x1, y1) (x2, y2) = (x1 + x2, y1 + y2)
```

```
sumarLista :: [Int] -> Int
```

```
sumarLista [] = 0
```

```
sumarLista (x:xs) = x + sumarLista xs
```

```
data Figura = Circulo Double |  
             Rectangulo Double Double
```

```
area :: Figura -> Double
```

```
area (Circulo r) = pi * r * r
```

```
area (Rectangulo b h) = b * h
```

```
miArea = area (Rectangulo 4 5)
```

```
obtenerCabeza :: [a] -> Maybe a
```

```
obtenerCabeza lista = case lista of
```

```
    [] -> Nothing
```

```
    (x:_) -> Just x
```

```
analizarCoordenada :: (Int, Int) -> String
```

```
analizarCoordenada punto = case punto of
```

```
    (0, 0) -> "En el origen"
```

```
    (x, y) -> "En otro punto del plano"
```

Control de flujo :: Expresión Condicional

El control de flujo por selección se implementa por medio de una expresión. Al tratarse de una expresión, retorna un valor.

`if condición then expresión_a else expresión_b`

Donde la condición debe ser de tipo Bool, y `expresión_a` y `expresión_b` deben ser exactamente del mismo tipo, el cual será el tipo final de toda la expresión condicional.

```
absoluto :: (Ord a, Num a) => a -> a
absoluto n = if n < 0 then -n else n

clasificarEdad :: (Ord a, Num a) => a -> [Char]
clasificarEdad edad = "La persona es " ++
    (if edad >= 18 then "mayor" else "menor")
```

Las expresiones condicionales son evaluadas con **evaluación perezosa**, lo que permite implementar funciones recursivas.

Control de flujo :: Uso de Guardas

Cuando se tienen múltiples condiciones en Haskell es mucho más idiomático utilizar Guardas (|) en lugar de anidar múltiples expresiones `if-then-else`. Esto mantiene el código mucho más limpio y legible.

```
clasificarNota :: (Ord a, Num a) => a -> String
clasificarNota nota
    | nota >= 9 = "Sobresaliente"
    | nota >= 7 = "Promocionado"
    | nota >= 4 = "Aprobado cursada"
    | otherwise = "Desaprobado"

diagnosticoMedico :: (Fractional a, Ord a) => a -> a -> String
diagnosticoMedico peso altura
    | imc < 18.5 = "Bajo peso"
    | imc < 25.0 = "Peso normal"
    | imc < 30.0 = "Sobrepeso"
    | otherwise = "Obesidad"
  where
    imc = peso / altura ^ 2
```

Control de Flujo :: Recursión

Haskell no brinda las sentencias de iteración habituales de los lenguajes imperativos convencionales (como while, do-while o for). La iteración fundamentalmente se realiza mediante la definición de funciones recursivas.

```
sumarUno :: Num a => [a] -> [a]
sumarUno [] = []
sumarUno (x:xs) = (x + 1) : sumarUno xs
```

Si bien la recursión explícita es la forma de repetición en el lenguaje, en la práctica se suele utilizar funciones de orden superior (como map, filter o folds como foldr y foldl) que ya traen encapsulados los patrones recursivos más comunes para recorrer, transformar o reducir estructuras de datos.

```
sumarUnoIdiomatico :: Num b => [b] -> [b]
sumarUnoIdiomatico xs = map (+1) xs
```

Mónadas

Como Haskell es puramente funcional, **una función no puede tener efectos colaterales.**

Sin embargo, para construir programas útiles se necesita, por ejemplo, leer archivos, bases de datos, inputs del usuario, o imprimir resultados por pantalla. Para lograr esto sin comprometer la pureza se utilizan *Mónadas*.

Una Mónada brinda un contexto que envuelve a un valor y permite realizar operaciones sobre ese valor.

La palabra reservada **do** inicia un bloque que te permite secuenciar operaciones monádicas

```
getContacto :: Int -> Maybe String
getContacto 1 = Just "user1@example.com"
getContacto _ = Nothing

presentarResultado :: Int -> IO ()
presentarResultado id = do
    putStrLn "Buscando información en el sistema..."
    let resultado = getContacto id
    case resultado of
        Just email -> putStrLn ("Contacto: " ++ email)
        Nothing    -> putStrLn "No se encontró el
                           contacto."

main :: IO ()
main = do
    putStrLn "Ingrese ID de usuario:"
    input <- getLine
    let id = read input
    presentarResultado id
```

Caso de Estudio: SML

Introducción

SML (Standard ML) es un lenguaje del paradigma funcional que cuenta con algunas características interesantes:

- Lenguaje **fuertemente tipado** con **tipado seguro**.
- Utiliza un mecanismo de **inferencia de tipos**.
- Polimorfismo paramétrico de manera natural.

Es un lenguaje utilizado en las primeras fases del desarrollo de compiladores, investigación y sistemas dedicados a la demostración de teoremas.

```
fun remove_duplicates [] = []  
  | remove_duplicates (x::xs) =  
    let  
      val rest = remove_duplicates xs  
    in  
      if List.exists (fn y => y = x) rest then rest  
      else x :: rest  
    end;  
  
val lista = remove_duplicates [1, 1, 1];
```

Introducción

Palabra reservada *fun*

Nombre de la función

Parámetros

Resultado

remove_duplicates
se define por
casos

Constructor para alcance
local (*let-in-end*)

```
fun remove_duplicates [] = []  
| remove_duplicates (x::xs) =  
  let  
    val rest = remove_duplicates xs  
  in  
    if List.exists (fn y => y = x) rest then rest  
    else x :: rest  
  end;
```

Llamada recursiva

Expresión condicional
con evaluación perezosa

función de orden superior
`exists : ('a -> bool) -> 'a list -> bool`

Identificadores y Ligaduras

El tipo asociado a un identificador se puede declarar de manera explícita o no hacerlo. En el segundo caso, el mecanismo de inferencia de tipos va a permitir ligar un tipo al identificador.

La declaración de variables se realiza usando la palabra reservada **val**.

SML	Output
<pre>val pi = 3.14; val pi: real = 3.14;</pre>	<pre>val pi = 3.14: real;</pre>
<pre>val radio = 2; val radio: int = 2;</pre>	<pre>val radio = 2: int;</pre>
<pre>val seguir = true; val seguir: bool = true;</pre>	<pre>val seguir = true: bool;</pre>
<pre>val letra = #"c"; val letra: char = #"c";</pre>	<pre>val letra = #"c": char;</pre>
<pre>val cadena = "hola"; val cadena: string = "hola";</pre>	<pre>val cadena = "hola": string;</pre>

Una variable es una abstracción de un valor.

El valor ligado al identificador no puede cambiar durante la ejecución del programa.

Identificadores y Ligaduras

La declaración de funciones se realiza usando la palabra reservada **fun**. El identificador (el nombre de la función) queda ligado a una expresión.

Las funciones tienen un sólo argumento. Funciones con más de un argumento son simuladas usando tuplas o funciones *curried*.

Las funciones definidas de manera curried van a permitir realizar **aplicaciones parciales**.

SML	Output
<pre>fun area r = r * r * pi; fun area r:real = r * r * pi; fun area (r:real) : real = r * r * pi;</pre>	<pre>val area = fn: real → real;</pre>
<pre>fun add x y = x + y; fun add(x, y) = x + y;</pre>	<pre>val add = fn: int → (int → int); val add = fn: int * int → int;</pre>
<pre>fun add x y:real = x + y; fun add x y = x + y: real;</pre>	<pre>val add = fn: real → (real → real);</pre>
<pre>fun add(x, y:real) = x + y; fun add(x, y) : real = x + y; fun add(x, y) = x + y : real;</pre>	<pre>val add = fn: real * real → real;</pre>

Identificadores y Ligaduras

Es posible definir un identificador local a una expresión usando la palabra reservada `let`.

`let` *declaración* **`in`** *expresión* **`end`**

Cuando termina la evaluación de la expresión, **las ligaduras de la declaración se descartan**.

SML	Output
<code>val m = 2;</code>	<code>val m = 2: int;</code>
<code>val r = let val m = 3; val n = 6; in m*n end;</code>	<code>val r = 18: int;</code>
<code>m;</code>	<code>val it = 2: int;</code>

Sistema de Tipos

El lenguaje provee los tipos simples *int*, *real*, *char*, *string* y *bool*, y tuplas y listas como tipos estructurados.

Una **tupla** permite agrupar valores que pueden ser de distintos tipos, mientras que una **lista** es una colección ordenada de elementos del mismo tipo.

SML	Output
<code>val t = (1, 2.0, "tres");</code>	<code>val t = (1, 2.0, "tres"): int * real * string;</code>
<code>val t = (("Lenguajes", 2023), (#"v", 2.0));</code>	<code>val t = (("Lenguajes", 2023), (#"v", 2.0)): (string * int) * (char * real);</code>
<code>val l = [1, 2, 3];</code>	<code>val l = [1, 2, 3]: int list;</code>
<code>val l = 1::[2, 3];</code>	<code>val l = [1, 2, 3]: int list;</code>
<code>val l = [[1, 2], [2, 3]];</code>	<code>val l = [[1, 2], [2, 3]]: int list list;</code>
<code>val l = [#"x", "hola", 2.0];</code>	Error de tipos

Sistema de Tipos

El lenguaje también permite al usuario definir sus propios tipos de datos, a través de la palabra reservada **datatype**. El usuario debe definir el nombre del nuevo tipo y los valores que pertenecen al tipo.

SML	Output
<pre>datatype color = Red Green Blue;</pre>	<pre>val Red = Red: color; val Green = Green: color; val Blue = Blue: color; datatype color = { con Red = Red: color; con Green = Green: color; con Blue = Blue: color; };</pre>
<pre>datatype tree = Void Node of char * tree * tree;</pre>	<pre>val Void = Void: tree; val Node = Node: char * tree * tree → tree; datatype tree = { con Void = Void: tree; con Node = Node: char * tree * tree → tree; };</pre>

Sistema de Tipos

ML tiene **operadores sobrecargados**, pero **no permite sobrecarga de funciones**.

SML	Output
<code>val x = 2 + 2;</code>	<code>val x = 4: int;</code>
<code>val xr = 2.0 + 3.0;</code>	<code>val xr = 4.0: real;</code>
<code>fun f x = x + 1;</code>	<code>val f = fn: int → int;</code>
<code>fun f x = x + 1.0;</code>	<code>val f = fn: real → real;</code>
<code>f 2</code>	Error de tipos
<code>f 2.0</code>	<code>val it = 3.0: real;</code>

Sistema de Tipos

Además, **no soporta expresiones mixtas ni incluye coerciones.**

SML	Output
<code>val x = 2 + 2.0;</code>	Error de tipos
<code>val x = 2 + Real.floor(2.0);</code>	<code>val x = 4: int;</code>
<code>val x = Real.fromInt 2 + 2.0;</code>	<code>val x = 4.0: real;</code>

Sistema de Tipos :: Inferencia de Tipos

El tipo asociado a un identificador es inferido a partir del contexto en el que aparece el identificador.

El **tipo de una variable** es inferido a partir del valor al cual fue ligada la variable, o bien a partir de las operaciones en las que interviene.

SML	Output
<code>val x = 2;</code>	<code>val x = 2: int;</code>
<code>val y = 3 * 4;</code>	<code>val y = 12: int;</code>

El **tipo de una función** es inferido a partir del tipo de sus argumentos y el tipo del resultado.

SML	Output
<code>fun sucesor x = x + 1;</code>	<code>val sucesor = fn: int → int;</code>
<code>fun sucesor x = x + 1.0;</code>	<code>val sucesor = fn: real → real;</code>

SML tiene una forma de tipado denominada **tipado seguro**.

El mecanismo de inferencia determina los tipos de las entidades del programa antes de la ejecución pero luego del chequeo de sintaxis.

Funciones

En ML, las funciones son consideradas **ciudadanos de primera clase**.

De esta manera, una función puede ser pasada como parámetro de otra función, y ser retornada como resultado de una función.

SML	Output
<code>fun f x = x + 1;</code>	<code>val f = fn: int → int;</code>
<code>fun add_func func x = func x + x;</code>	<code>val add_func = fn: (int → int) → int → int;</code>
<code>add_func f 10;</code>	<code>val it = 21: int;</code>

Funciones

Las funciones en ML tienen un único argumento. Funciones con más de un argumento son simuladas usando tuplas como argumento o con funciones curried.

Las **funciones curried** permiten definir nuevas funciones a partir de **aplicaciones parciales**.

SML	Output
<code>fun mayor x y = x > y;</code>	<code>val mayor = fn: int → (int → bool);</code>
<code>val mayor_1 = mayor 1;</code>	<code>val mayor_1 = fn: int → bool;</code>
<code>val v = mayor 2 3;</code>	<code>val v = false: bool;</code>
<code>val v2 = mayor_1 3;</code>	<code>val v2 = false: bool;</code>

Funciones

Las funciones en ML tienen un único argumento. Funciones con más de un argumento son simuladas usando tuplas como argumento o con funciones curried.

En el caso de utilizar una **tupla como argumento**, todos los valores de la tupla deben tener un valor ligado antes de poder evaluar la función.

SML	Output
<code>fun mayor(x, y) = x > y;</code>	<code>val mayor = fn: int * int → bool;</code>
<code>val v = mayor(2, 3);</code>	<code>val v = false: bool;</code>
<code>val v = mayor(2);</code>	Error de tipos

Funciones :: Funciones Polimórficas

En SML, el tipo de las funciones polimórficas es un **politipo**, esto es, un tipo que contiene **variables de tipo**.

SML	Output
<pre>fun reverse [] = [] reverse (h::t) = (reverse t) @ [h];</pre>	<pre>val reverse = fn: $\forall$ 'a . 'a list $\rightarrow$ 'a list;</pre>
<pre>reverse [1, 2, 3];</pre>	<pre>val it = [3, 2, 1]: int list;</pre>
<pre>reverse ["a", "b", "c"];</pre>	<pre>val it = ["c", "b", "a"]: char list;</pre>
<pre>reverse [];</pre>	<pre>val it = []: 'a list;</pre>
<pre>fun map f [] = [] map f (h::t) = [f h] @ map f t;</pre>	<pre>val map = fn: $\forall$ 'a 'b . ('a $\rightarrow$ 'b) $\rightarrow$ 'a list $\rightarrow$ 'b list;</pre>

Politipo con dos variables de tipo

Control de flujo :: Pattern Matching

Una de las principales características de ML es el uso de **pattern matching**.

El mecanismo de pattern matching permite ligar una variable a un valor en una estructura más compleja, además de controlar el flujo de la ejecución de un programa.

SML	Output
<pre>val tupla = (("Lenguajes", 2023), (#"v", 2.0)); val ((_, _), (r, _)) = tupla;</pre>	<pre>val r = #"v": char;</pre>
<pre>fun factorial 0 = 1 factorial x = x * factorial (x-1);</pre>	<pre>val factorial = fn: int → int;</pre>
<pre>fun length [] = 0 length (h::t) = 1 + length t;</pre>	<pre>val length = fn: ∀ 'a . 'a list → int;</pre>
<pre>fun f (0, _) = 0 f (0, "lenguajes") = 1 f (x, _) = 2;</pre>	<pre>val f = fn: int * string → int;</pre>

Control de flujo :: Expresión Condicional

El control de flujo por selección es implementado por medio de una expresión. Al tratarse de una expresión, retorna un valor.

if *condición* **then** *expresión_a* **else** *expresión_b*

- *condición* debe ser de tipo **bool**.
- *expresión_a* y *expresión_b* deben ser del mismo tipo, que será el tipo de la expresión condicional.

Las expresiones condicionales son evaluadas con **evaluación perezosa**, lo que permite implementar funciones recursivas.

SML	Output
<pre>fun factorial x = if x = 0 then 1 else x * factorial (x-1);</pre>	<pre>val factorial = fn: int → int;</pre>

Control de flujo :: Recursión

SML no brinda las sentencias de iteración habituales de los lenguajes convencionales (while, do-while, for). La única manera de iterar es mediante la definición de funciones recursivas.

SML	Output
<pre>fun factorial 0 = 1 factorial x = x * factorial (x-1);</pre>	<pre>val factorial = fn: int → int;</pre>
<pre>fun length nil = 0 length (h::t) = 1 + length t; length [1, 2, 3];</pre>	<pre>val length = fn: ∀ 'a . 'a list → int; val it = 3: int;</pre>
<pre>fun reverse nil = [] reverse (h::t) = (reverse t) @ [h]; reverse [1, 2, 3];</pre>	<pre>val reverse = fn: ∀ 'a . 'a list → 'a list; val it = [3, 2, 1]: int list;</pre>

¿Preguntas?
